

Drone Garage

3/5/2026

Improved Metric Measurements

- For some reason the painless mesh library doesn't scale its mesh times to milliseconds but rather **microseconds**
- **Heartbeats** have a latency of 1 second (by design one is sent each second)
- **Broadcasts** have latency values ranging from 15ms to 40ms (mostly around 20ms)
- **MAVLink UART Flood messages** are about 50ms

```
@@{"type":"mesh_meta","event":"heartbeat","nodeName":"node2","meshId":3,"leaderId":3,"uptime":81291}
[node2] myMeshId=3 leader=3 me? yes lastHB=81299 now=82278
[painless compare] lastHB=81299271 now=82278445 latency(millis)=979 latency(painless)=979178

@@{"type":"mesh_meta","event":"heartbeat","nodeName":"node2","meshId":3,"leaderId":3,"uptime":82281}
[tp] RX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[tp] TX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[tp] RX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[tp] TX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[node2] myMeshId=3 leader=3 me? yes lastHB=82289 now=83278
[painless compare] lastHB=82289453 now=83282176 latency(millis)=993 latency(painless)=992727

@@{"type":"mesh_meta","event":"heartbeat","nodeName":"node2","meshId":3,"leaderId":3,"uptime":83291}
[tp] RX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[tp] TX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[node2] myMeshId=3 leader=3 me? yes lastHB=83299 now=84278
[painless compare] lastHB=83299220 now=84282359 latency(millis)=983 latency(painless)=983143

[rx] type=ctr len=58
  ctl heartbeat from 2
  sent=48280831 rcv=48295415 latency=14584 learned/updated
[node2] myMeshId=3 leader=2 me? no lastHB=18871 now=19276
[painless compare] lastHB=1276354 now=48701336 latency(millis)=48701336
[tp] RX: 1.0 msg/s, 58.0 B/s (0.46 kbps), payload 0.0 B/s
[tp] TX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[node2] rx from 3664036618: 58 bytes
[rx] type=ctr len=58
  ctl heartbeat from 2
  sent=49280806 rcv=49298378 latency=17572 learned/updated
[node2] myMeshId=3 leader=2 me? no lastHB=19874 now=20276
[painless compare] lastHB=1276354 now=49701315 latency(millis)=49701315
[tp] RX: 1.0 msg/s, 58.0 B/s (0.46 kbps), payload 0.0 B/s
[tp] TX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[node2] rx from 3664036618: 58 bytes
[rx] type=ctr len=58
  ctl heartbeat from 2
  sent=50277003 rcv=50296199 latency=19196 learned/updated
[node2] myMeshId=3 leader=2 me? no lastHB=20871 now=21276
```

Fine Tuning the UART Flood

- UART Flood values
 - 50ms $\Rightarrow$ way too fast, causes the board to restart, messing with the leadership and the mesh as a whole
 - 50 bytes every 50ms $\Rightarrow$ crashes
 - 20 bytes every 100ms $\Rightarrow$ works $\Rightarrow$ 1.60 Kbps tx $\Rightarrow$ maxing out everything crashes 9 minutes in $\Rightarrow$ well never hit that data so doesnt really matter too much
 - **The RX was only .03 kbps less than the TX so it can clearly handle it**
- Several text files with these measurements exist –
mesh_gateway_putty_flood5.txt contains the latest results

Bit Error Rate

- `bit_err_test.py`
- Set baud rate on the arduino ide serial monitor to 57600 and the file's input baud rate to 57600
 - Prevents clipping of sent bytes
 - Was sending consistently incorrect data
- Possibly lower the baud rate even more (115200 was too high) – just keep it at 57600
- **`bit_err_putty1.txt`** contains the latest results

Improved Metric Measurements

- Drop Rate
 - Take screenshots on sender and receiver(s) (requires 2+ ppl) (real quick)

Possible Fixes/Improvements

- Should prob try and find a way to improve our mesh's rates
 - Possible solutions include
 - Not doing any computation on the ESPs that could be instead done on the Pis
 - Getting rid of prints or scheduled tasks that take up resources

To Text File (Real Quick)

- I think puTTY or whatever should be able to have a serial monitor and can put whats printed there to a text file
 - <https://putty.org/index.html>
- Open puTTY > Session (on the left menu) > Logging > select “Printable Output” > Session > Select file name and location > Set to “Serial” on the top > Set to whatever COM the ESP is on > Set the baud rate to 115200
 - Make sure the ESP is plugged in
 - U also have to do this every time

Drone GPS Data Outside

- Because of building location and concrete cant quite get a ton of gps data rn
- In the future, gonna try having an ethernet connection for longer connections
- In the files loc.log and loc2.log (loc3.log the connection was severed)
 - Using router and stuff
 - Latitude and longitude values were accurate, matching phone GPS data and correctly changing upon relocation
- Next time, using router + (lab) phone via ethernet on Stevens IOT
-

More Network Experiments (3 Node, etc)

- Message Relay – Node to Node to Node
 - Relay mavlink data
- Experiments
 - 3 Node w/o RF blocker
 - 3 Node w/ RF blocker
 - Note: Ask Ishan where the blocker is
 - Others?
 - I recall a conversation w Ishan where he showed me a device in the lab to mess with frequencies and stuff
 - Experimenting with distance, movement, etc

More Network Experiments

- Message Relay

- `painlessmesh::mesh::sendSingle(nodeID, msg)`
 - Apparently it naturally does a relay if it cant see it or something?
 - Apparently **sendBroadcast() already does a relay** – what now?
 - <https://randomnerdtutorials.com/esp-mesh-esp32-esp8266-painlessmesh/>
 - Is there something we want to do for single messages? Perhaps test latency for singles?
I mean if we already measure latency on receiver nodes does that matter?
 - Whatever just gonna focus on making sure that the system can relay the MAVLINK info

Camera Data Sending Throughout the Mesh

- Need to set up the PI 4 – transfer files with ssh using: `ssh pi@10.155.37.195`, whose password is pi
 - Pi 4 kinda buns cuz torch wont pip install: switching `/etc/apt/sources.list.d/debian.sources` stuff trixie -> forky
 - Had to reflash so its now dronegarage with password dronegarage
 - Note to self: the object detection code requires pytorch which must be on a
 - Pip Installing Torch leads to an “Out of space” error
 - Trying pip install with `--no-cache-dir`
 - Almost worked
 - Changing `TMPDIR=~/.tmp` ????
 - Installation command becomes `TMPDIR=~/.tmp pip install opencv-python ultralytics paho-mqtt numpy`
 - Works i think
 - Ayy ill take it
- Camera setup:
 - Not appearing in `sudo raspi-config` → automatically detected as long as plugged in (`picamera-still` or whatever the command is works)
 - Problem: `cv2.VideoCapture()` wont work (no matter what fields (tried a bunch))
 - Perhaps `picamera2` will work? **Works**
 - Must pip install `rpi-libcamera` `rpi-kms` `picamera` and make sure venv has `--system-site-packages` so that the venv can use `picamera` stuff
 - Also u show up blue initially so